
IIC2523

Sistemas Distribuidos

— Hernán F. Valdivieso López —
(2026 - 1 / Clase 22)

Redes *Peer to Peer* (P2P) Estructuradas

¿Qué hacer cuando todos son iguales?

Temas de la clase

1. *Peer to Peer* Estructurado
 - a. *Distributed Hash Table* (DHT)
 - b. Seguridad en DHT
 - c. Caso aplicado - *Chord*

Peer to Peer (P2P) - Seguridad inherente (clase anterior)

En una red P2P no estructurada, donde cada cliente también actúa como servidor y la información está distribuida, un atacante se enfrenta a varias dificultades:

- ◆ No sabe de antemano qué nodo posee la información que desea interceptar o atacar.
- ◆ Bloquear o comprometer un nodo no afecta al funcionamiento global del sistema.
- ◆ Cada nodo puede usar mecanismos de defensa independientes (cifrado, *firewall*, IDS/IPS...), lo que impide una estrategia de ataque uniforme.
- ◆ Si la información está fragmentada o replicada, atacar requiere coordinar acciones contra múltiples nodos, aumentando el costo y reduciendo la probabilidad de éxito.

Peer to Peer (P2P) - Seguridad inherente (clase anterior)

En una red P2P no estructurada, donde cada cliente también actúa como servidor y la información está distribuida, un atacante se enfrenta a varias dificultades:

◆ No sabe de antemano qué nodo posee la información que desea interceptar o

Sin embargo... la libertad total en las conexiones y búsquedas tiene un costo:

◆ **La comunicación y la localización de información pueden ser lentas, ruidosas y poco eficientes.** 😓

◆ Si la información está fragmentada o replicada, atacar requiere coordinar acciones contra múltiples nodos, aumentando el costo y reduciendo la probabilidad de éxito.

Peer to Peer (P2P) - Seguridad inherente (clase anterior)

En una red P2P no estructurada, donde cada cliente también actúa como servidor y la información está distribuida, un atacante se enfrenta a varias dificultades:

◆ ~~No sabe de antemano qué nodo posee la información que desea intercambiar o~~

Sin embargo... la libertad total en las conexiones y búsquedas tiene un costo:

◆ La comunicación y la localización de información pueden ser lentas, ruidosas y poco eficientes. 🥵

¿Existe una forma de organizar la red, sin servidores centrales, que mantenga la descentralización pero optimice la búsqueda de datos? 🤔

éxito.

Peer to Peer Estructurado

Distributed Hash Table (DHT)

Seguridad en DHT

Caso particular: Chord

Distributed Hash Table (DHT)

- ◆ Estructura de datos distribuida que permite almacenar y buscar eficientemente sin servidor central.
- ◆ Se utiliza una función de **hash consistente** para definir una *key* para cada dato y nodo de la red.
 - ◆ La *key* del nodo (o ID) suele ser el *hash* de una pieza de información de identificación sobre el nodo, como su dirección IP, correo, etc.
 - ◆ Se utiliza la *key* para determinar la posición del dato en la red y quién es el nodo responsable de dicho dato.

Distributed Hash Table (DHT) - Hash consistente

- ◆ Primero veamos *hash* "normal" (no consistente)
 - ◆ Asumiendo 5 nodos, se tendría que hacer $\text{hash}(X) \% 5$ para determinar qué nodo guarda la información de X.
 - ◆ Si un nodo se sale, habría que hacer $\text{hash}(X) \% 4$ y todos los datos cambiarían al nodo que es responsable de él.

Distributed Hash Table (DHT) - Hash consistente

- ◆ Primero veamos *hash* "normal" (no consistente)
 - ◆ Asumiendo 5 nodos, se tendría que hacer $\text{hash}(X) \% 5$ para determinar qué nodo guarda la información de X.
 - ◆ Si un nodo se sale, habría que hacer $\text{hash}(X) \% 4$ y todos los datos cambiarían al nodo que es responsable de él.
- ◆ Ahora, con *hash* consistente.
 - ◆ Este entrega un valor que ya aplica un $\%N$ internamente que no depende del tamaño de la red → El *hash* está condicionado a un rango predefinido.
 - ◆ El *hash_consistente(id_nodo)* dará la posición del nodo en el rango predefinido.
 - ◆ Si se va un nodo, a lo más el nodo "sucesor" y "predecesor" serán afectados.

Distributed Hash Table (DHT) - Hash consistente

Ejemplo

- ◆ Hash da valores entre 0 y 9.
- ◆ Tengo 3 nodos cuyo Hash es 0, 4 y 8 respectivamente.
 - ◆ Nodo 0 → Responsable de todos los datos cuya *key* es 9 y 0.
 - ◆ Nodo 4 → Responsable de todos los datos cuya *key* es 1, 2, 3 y 4.
 - ◆ Nodo 8 → Responsable de todos los datos cuya *key* es 5, 6, 7 y 8.

Distributed Hash Table (DHT) - Hash consistente

Ejemplo

- ◆ Hash da valores entre 0 y 9.
- ◆ Tengo 3 nodos cuyo Hash es 0, 4 y 8 respectivamente.
 - ◆ Nodo 0 → Responsable de todos los datos cuya *key* es 9 y 0.
 - ◆ Nodo 4 → Responsable de todos los datos cuya *key* es 1, 2, 3 y 4.
 - ◆ Nodo 8 → Responsable de todos los datos cuya *key* es 5, 6, 7 y 8.
- ◆ Se va el nodo 8.
 - ◆ Nodo 0 adjunta la información del nodo 8. Es decir, será responsable de 5, 6, 7, 8, 9 y 0.
 - ◆ Nodo 4 sigue sin cambios.

Distributed Hash Table (DHT) - Funcionamiento

- ◆ El *Hash* produce una salida numérica de longitud fija.
 - ◆ Por ejemplo, $\text{SHA-256}(\text{uwu.mp4}) = 192371391873192381444491831911$
- ◆ Asignación a un nodo cuyo identificador esté más próximo al *hash* (según alguna métrica, como XOR, prefijo, o distancia circular).

Distributed Hash Table (DHT) - Funcionamiento

- ◆ El *Hash* produce una salida numérica de longitud fija.
 - ◆ Por ejemplo, SHA-256(uwu.mp4) = 192371391873192381444491831911
- ◆ Asignación a un nodo cuyo identificador esté más próximo al *hash* (según alguna métrica, como XOR, prefijo, o distancia circular).
- ◆ Cada nodo tiene una tabla de *routing* que corresponde a un conjunto de punteros a otros nodos, cuidadosamente elegidos, para permitir que un nodo pueda encaminar solicitudes hacia el nodo responsable de una clave, en pocos saltos.
- ◆ **Si un atacante logra que todas las entradas de esa tabla apunten a nodos que él controla, ahí el nodo víctima queda atrapado.**

Distributed Hash Table (DHT) - Seguridad

1. Difícil secuestrar rutas sin alterar muchos nodos

- ◆ Cada nodo tiene una posición y vecinos determinados por su ID y el *hash*.
- ◆ Las rutas de búsqueda siguen ciertas reglas determinadas por el algoritmo.

Implicancia

- ◆ Para un atacante, no basta con insertar uno o dos nodos maliciosos en la red; necesitaría estar posicionado en ubicaciones específicas a lo largo de múltiples rutas para tener una probabilidad razonable de interceptar búsquedas.
- ◆ Permite mitigar ataques Eclipse.

Distributed Hash Table (DHT) - Seguridad

2. Nodos no pueden elegir qué datos almacenar

- ◆ Cada dato se almacena en los nodos cercanos al *hash* de su clave.
- ◆ Los nodos responsables no eligen los datos; es una consecuencia del algoritmo de ubicación.

Implicancia

- ◆ Un nodo malicioso no puede simplemente decidir almacenar claves críticas o espiar datos sensibles a no ser que logre conseguir un ID específico y para eso debe "vencer" al *hash*.

Distributed Hash Table (DHT) - Seguridad

3. La red se adapta a nodos que entran/salen

- ◆ Están diseñadas para ser dinámicas: si un nodo desaparece, los datos se replican o redistribuye automáticamente.
- ◆ Las tablas con la posición de los nodos se actualizan con el tiempo.

Implicancia

- ◆ La red es resiliente a ataques de denegación parcial, donde se intenta eliminar nodos clave.
- ◆ También permite expulsar nodos erráticos o sospechosos mediante rotación natural.

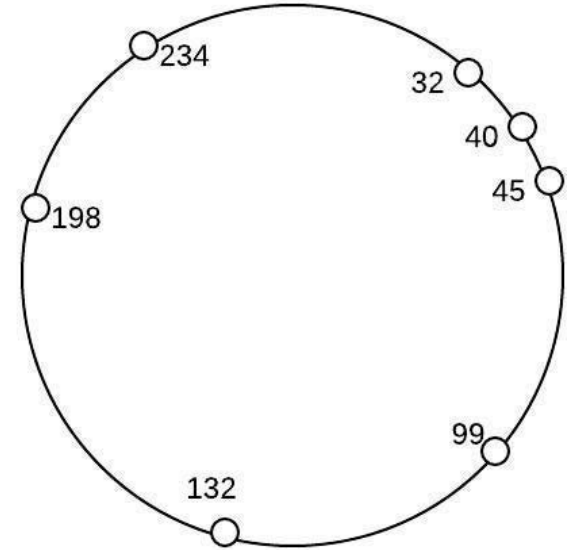
Distributed Hash Table (DHT) - Seguridad

Resumen: Aumenta el costo del comportamiento malicioso

- ◆ El esfuerzo para atacar de forma eficaz una DHT es mucho mayor que en una red no estructurada.
 - ◆ No puedes elegir tus vecinos libremente ni qué datos guardar en la tabla de punteros.
 - ◆ No puedes posicionarte arbitrariamente.
- ◆ Obliga a los atacantes a crear ataques masivos, coordinados y sofisticados, elevando el umbral técnico y económico.

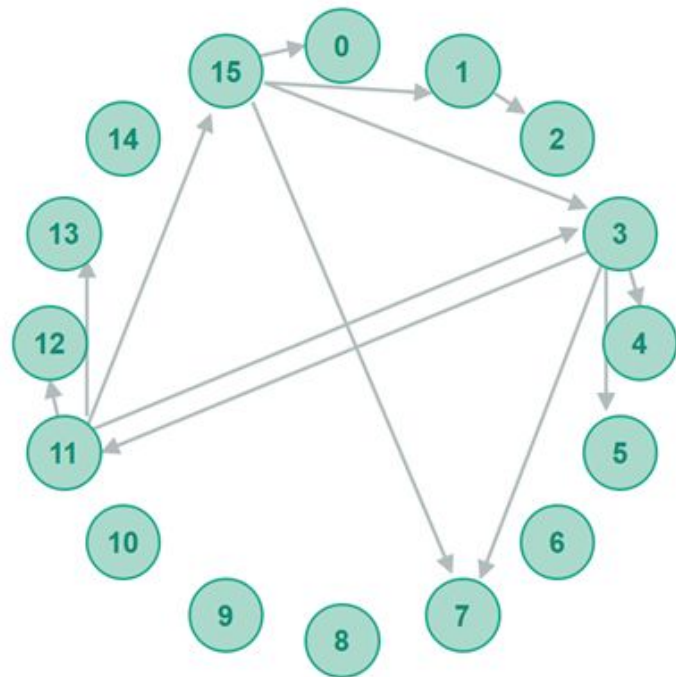
Distributed Hash Table (DHT) - Chord

- ◆ Se utiliza un DHT donde los nodos se organizan lógicamente en un anillo.
- ◆ Un elemento de datos con una clave k se asigna al nodo con el identificador más pequeño cuyo $\text{hash}(\text{id_nodo}) \geq k$.
 - ◆ Este nodo se conoce como el sucesor de k .



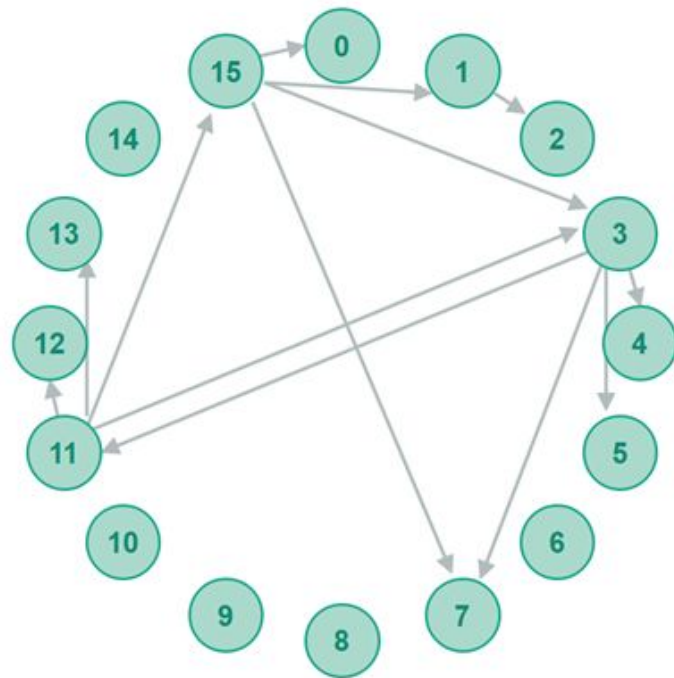
Distributed Hash Table (DHT) - Chord

- ◆ Se utiliza un DHT donde los nodos se organizan lógicamente en un anillo.
- ◆ Un elemento de datos con una clave k se asigna al nodo con el identificador más pequeño cuyo $\text{hash}(\text{id_nodo}) \geq k$.
 - ◆ Este nodo se conoce como el sucesor de k .
- ◆ Cada nodo contiene una *Finger Table* que corresponde a su tabla de enrutamiento.
- ◆ La tabla se construyen de manera que la longitud de la ruta más corta entre cualquier par de nodos sea óptima.
 - ◆ Orden $O(\log N)$ con N igual al tamaño de la red.

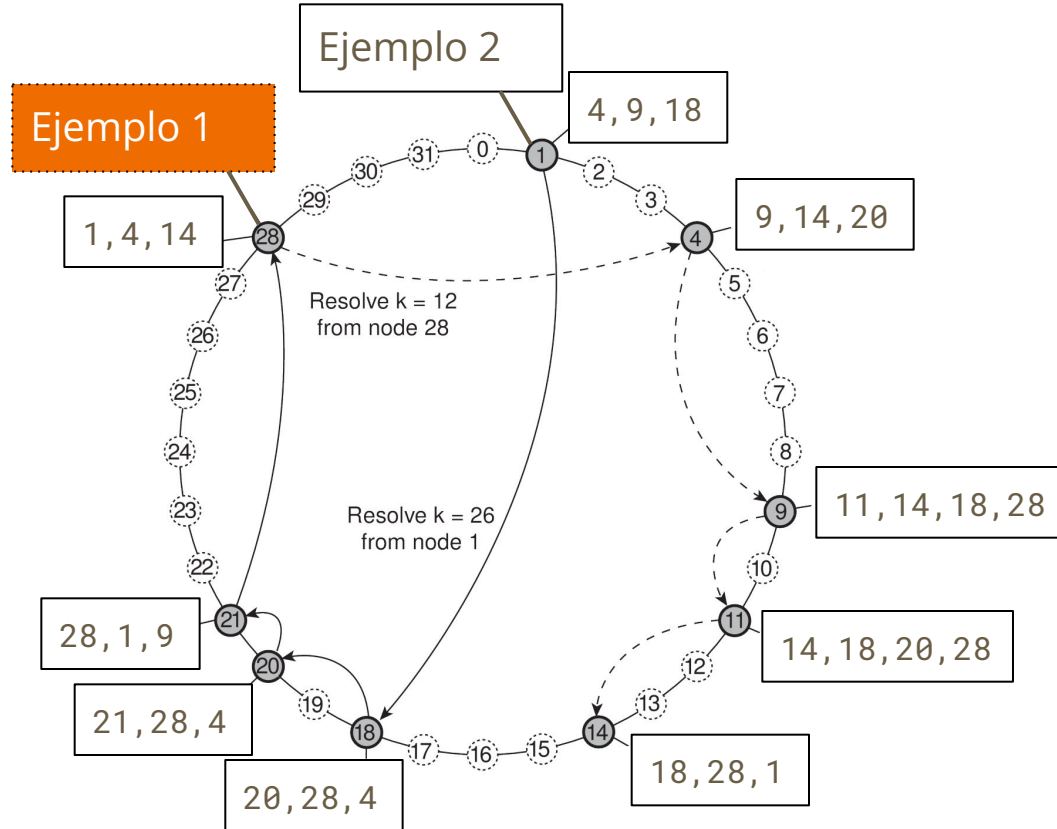


Distributed Hash Table (DHT) - Chord - Búsqueda

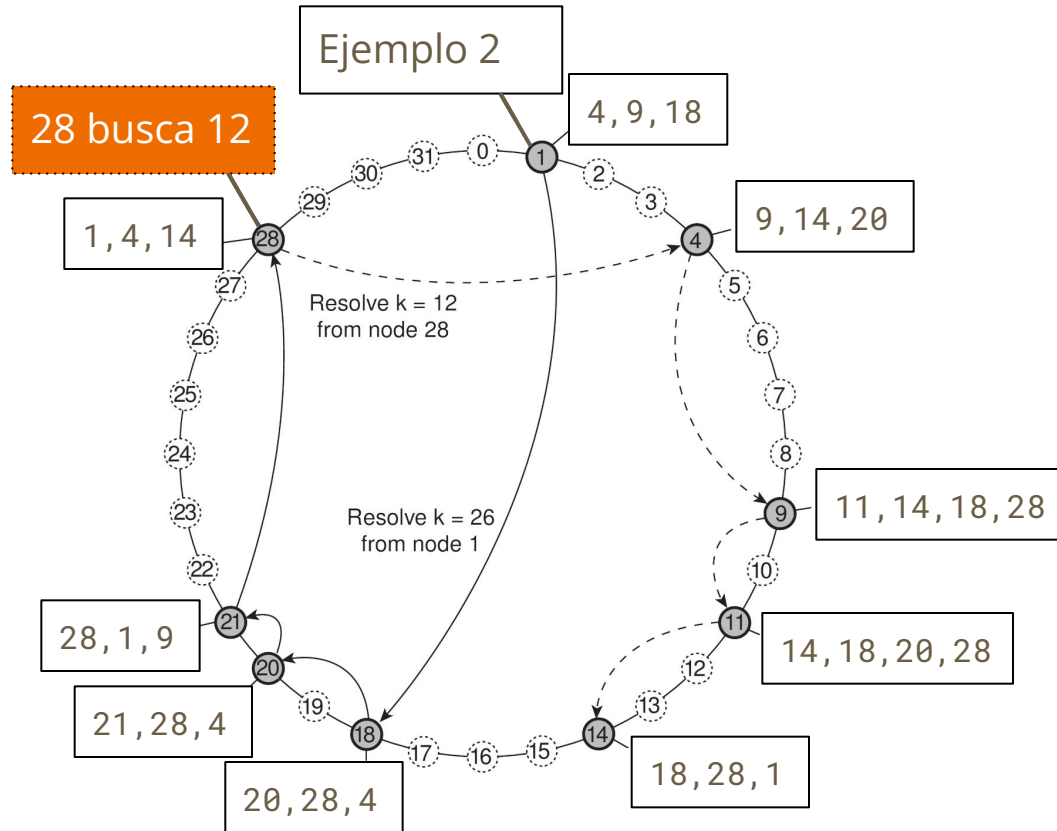
- ◆ Al momento de buscar una *key*, se busca el nodo más cercano **que no supere la *key***.
 - ◆ Aunque, si el nodo que busca ya está asociado una *key* mayor a la buscada, debe darse la vuelta en el anillo.
- ◆ Caso ideal: Llegar al nodo cuya *key* es igual a la buscada.
- ◆ Caso realista: Se llega al nodo cuya *key* es la más cercana y él determina si existe el nodo deseado o debe "pasarse" para buscar la información con el sucesor de la *key*.



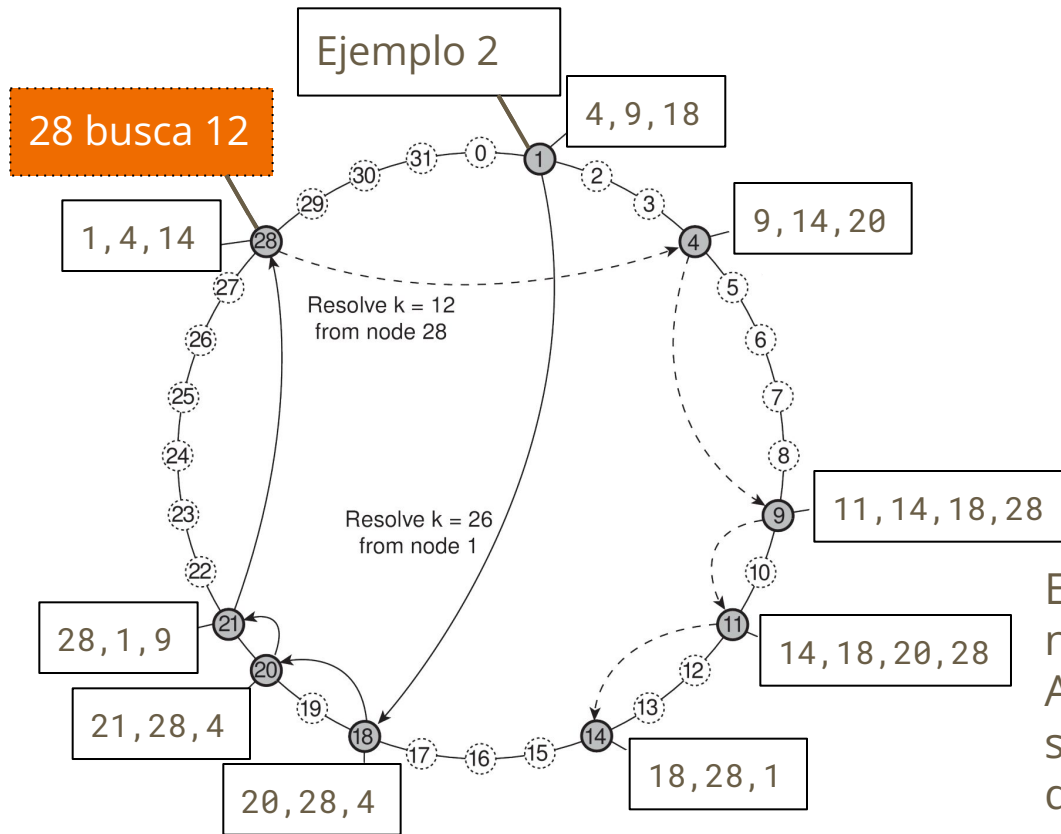
Distributed Hash Table (DHT) - Chord - Búsqueda



Distributed Hash Table (DHT) - Chord - Búsqueda

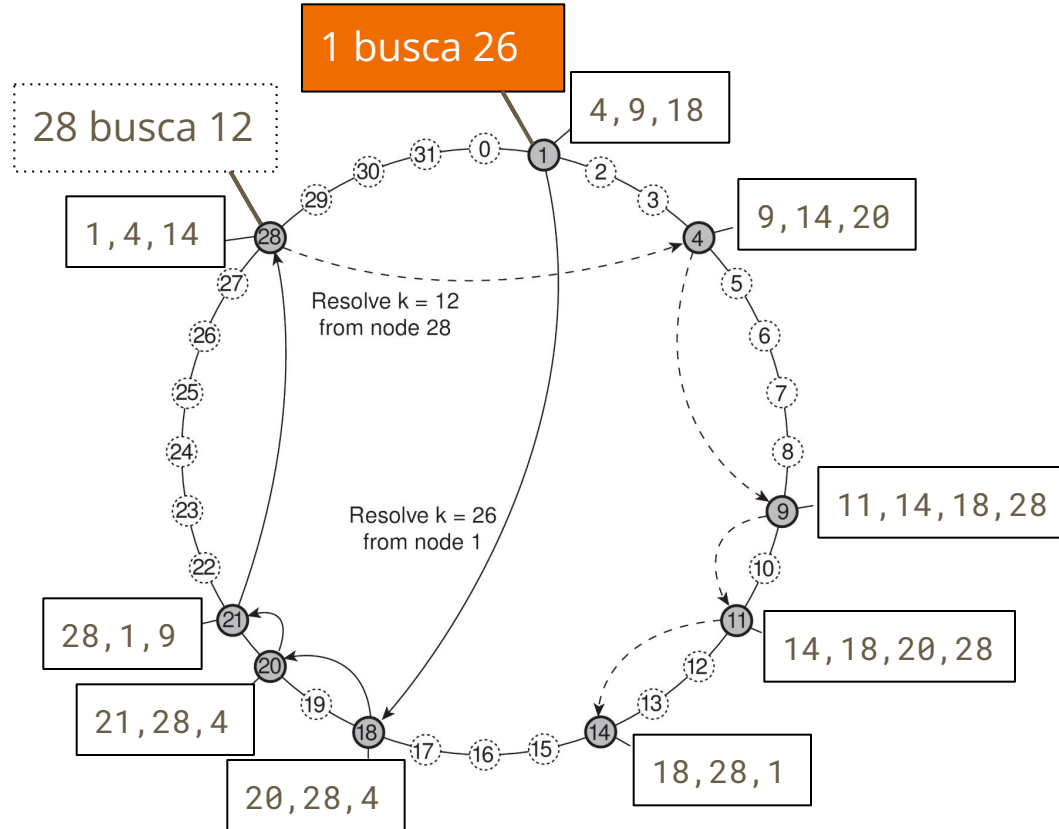


Distributed Hash Table (DHT) - Chord - Búsqueda



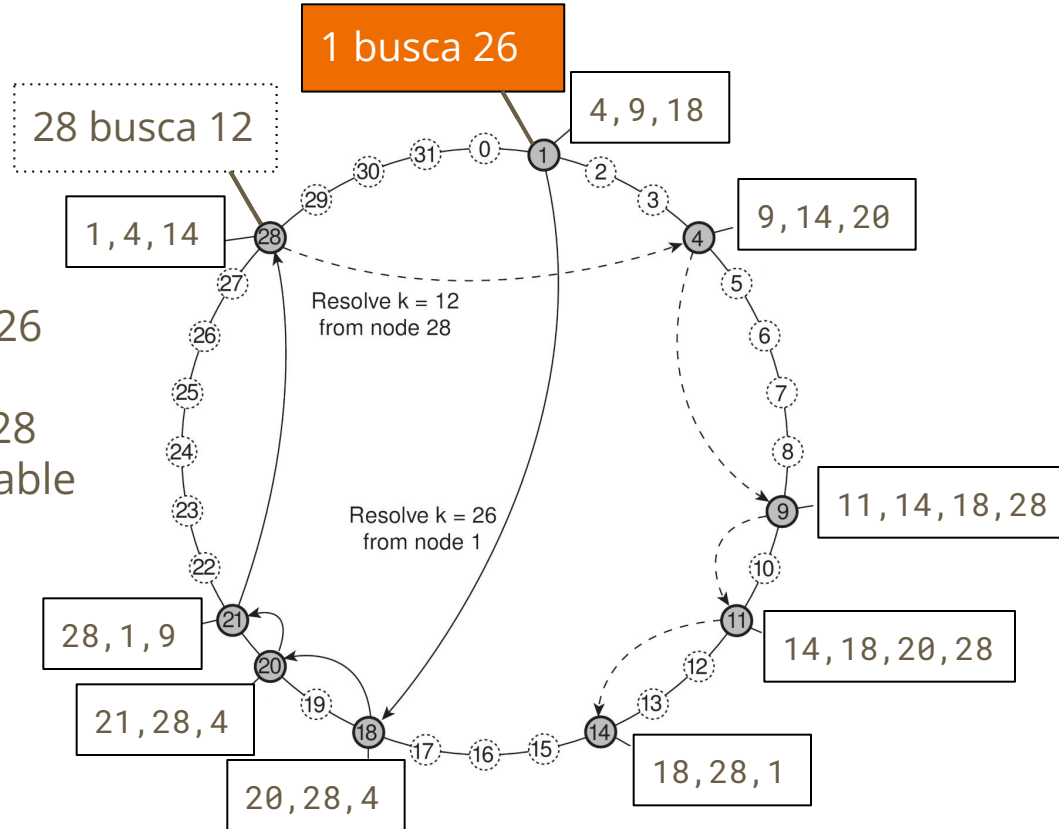
El nodo con key 12 no existe.
Así que el nodo 14 se hace responsable de sus datos.

Distributed Hash Table (DHT) - Chord - Búsqueda



Distributed Hash Table (DHT) - Chord - Búsqueda

El nodo con key 26
no existe.
Así que el nodo 28
se hace responsable
de sus datos.



Distributed Hash Table (DHT) - Chord - Churn

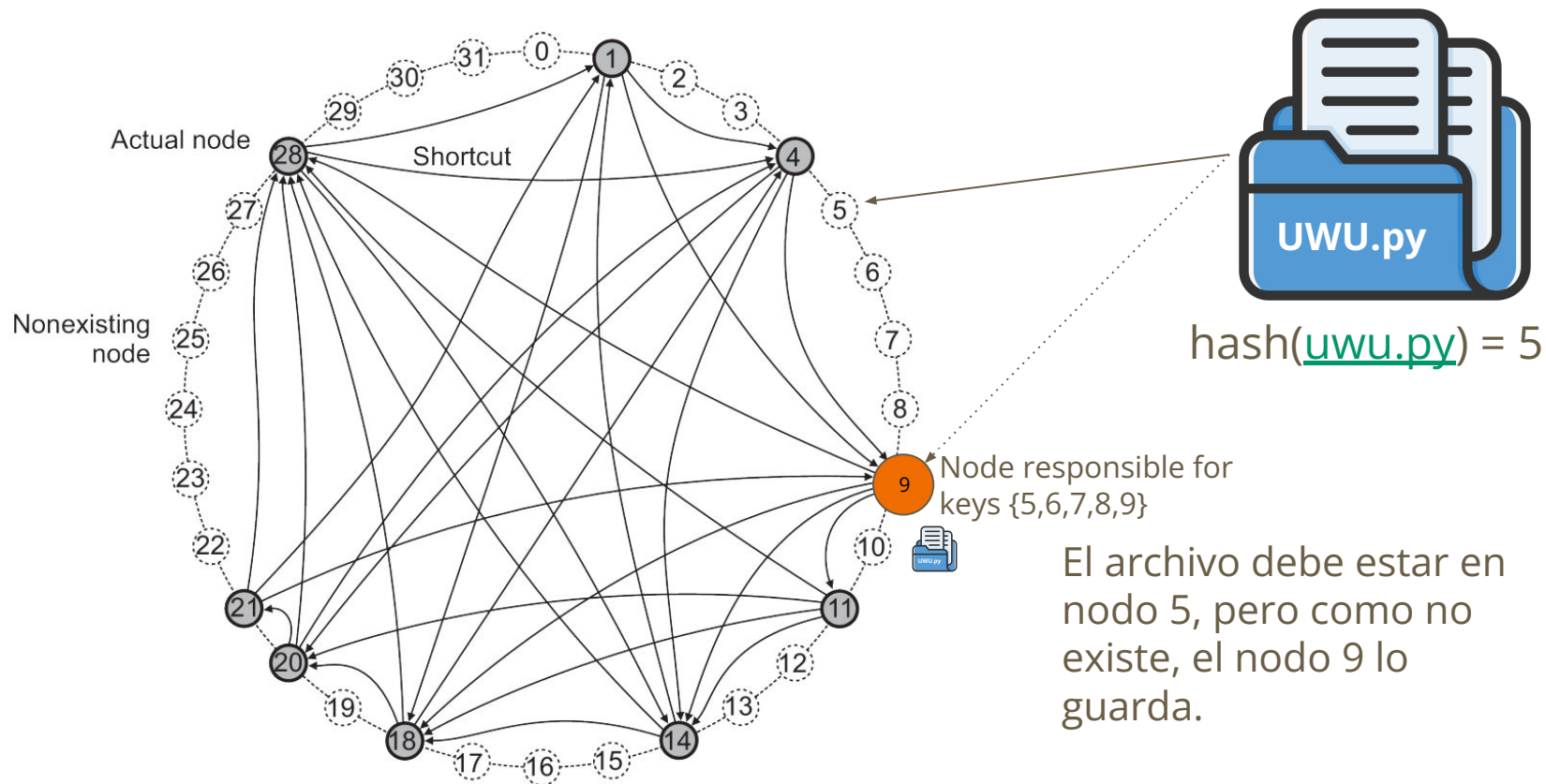
Entrada de nodo

- ◆ Cuando un nuevo nodo se une a la superposición, obtiene los datos necesarios para construir su *finger table* y el estado requerido de los miembros existentes a partir de su sucesor.
- ◆ El nodo se inserta en la posición que corresponda según su *key* ($\text{hash}(\text{id_nodo})$), y los datos que él se debe responsabilidad se transfieren.

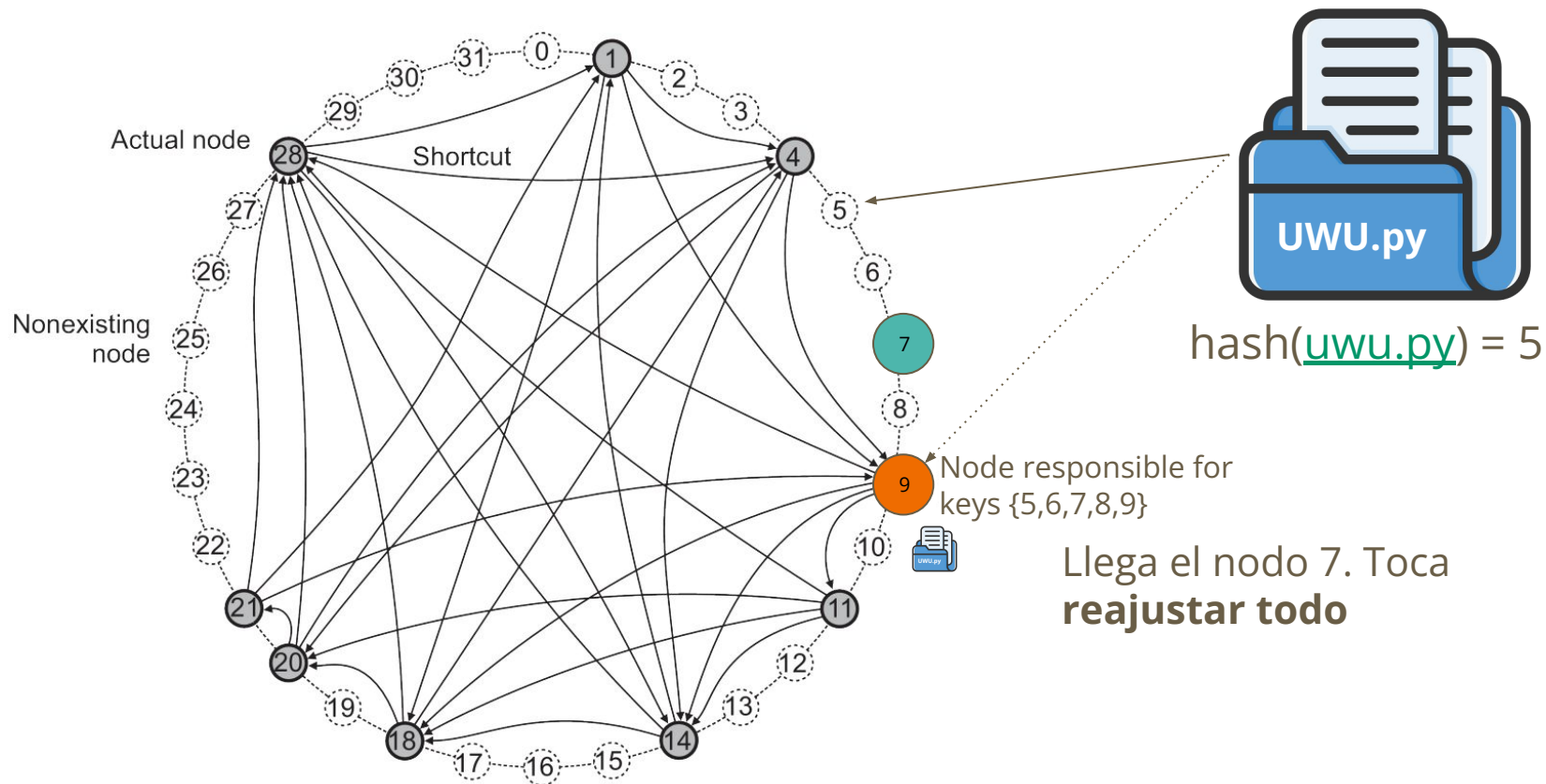
Salida salida de nodo

- ◆ Si un nodo falla o sale (voluntaria o involuntariamente), los nodos predecesores puede detectar su ausencia y reconfigurar su tabla para reflejar los cambios requeridos.

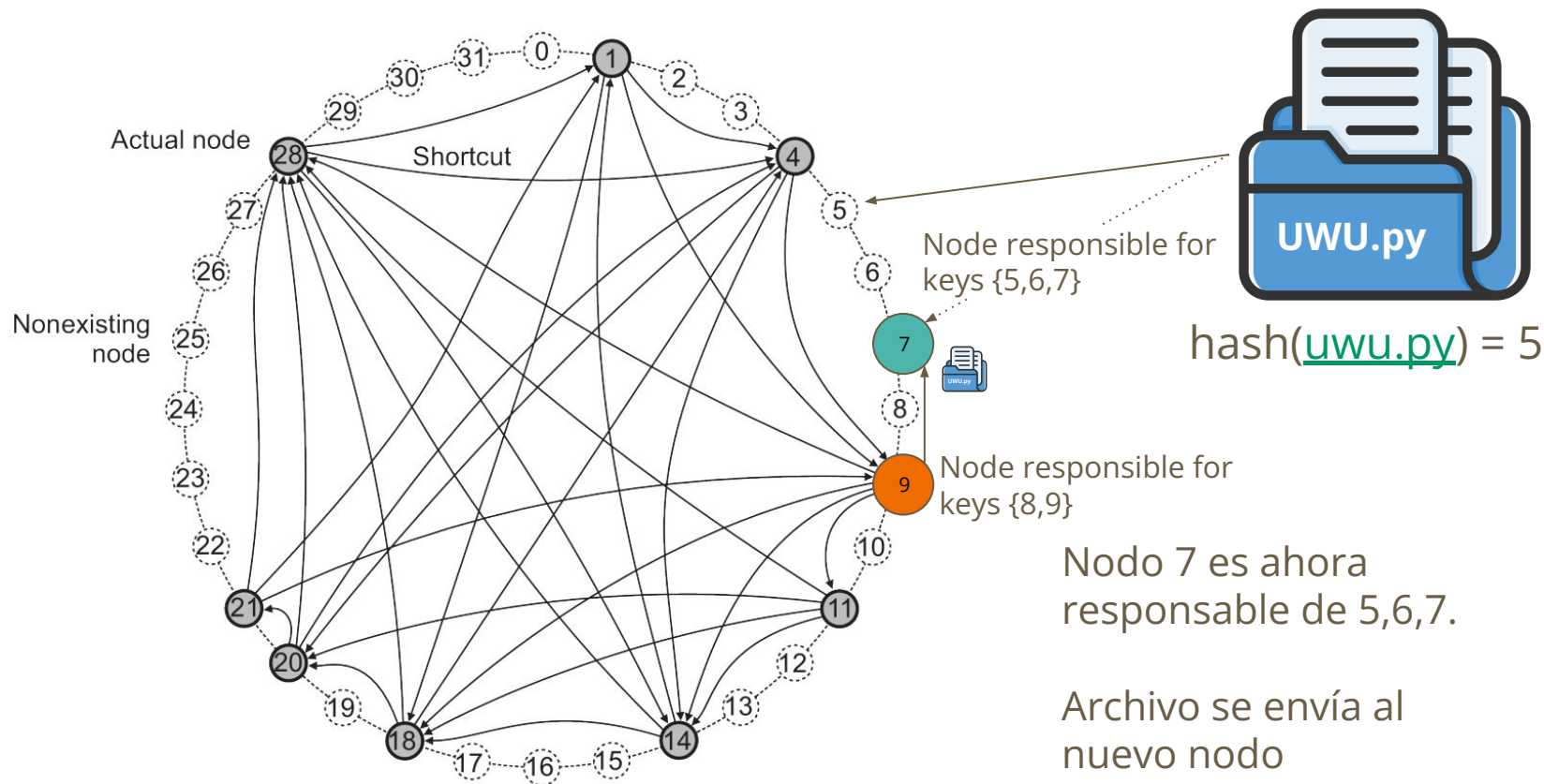
Distributed Hash Table (DHT) - Chord - Churn



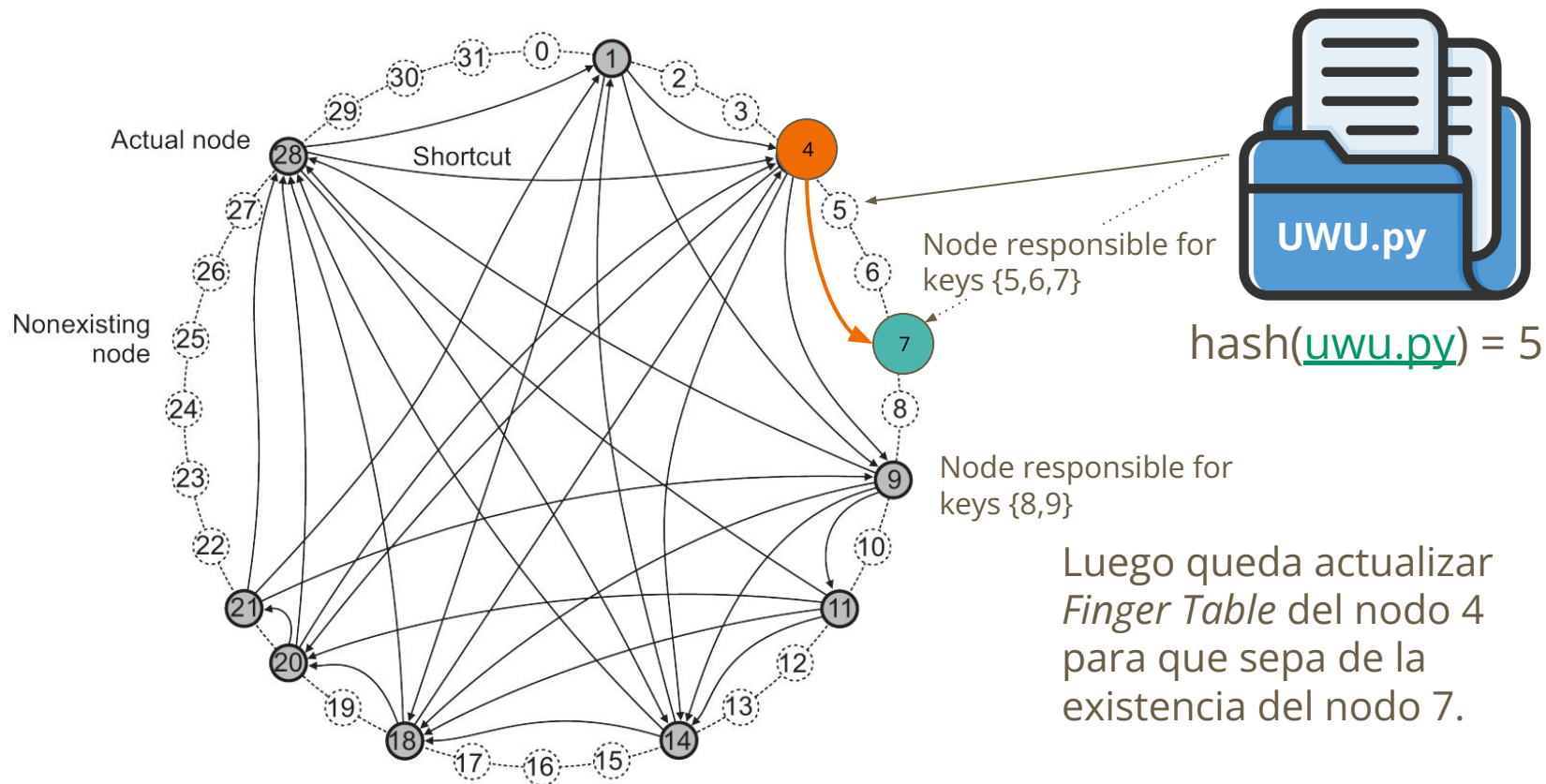
Distributed Hash Table (DHT) - Chord - Churn



Distributed Hash Table (DHT) - Chord - Churn



Distributed Hash Table (DHT) - Chord - Churn



Distributed Hash Table (DHT) - Chord - Seguridad

Chord hereda los aspectos de seguridad de una DHT.

1. Difícil secuestrar rutas sin alterar muchos nodos

***Finger tables* y rutas verificables**

- ◆ Las rutas están definidas por los *finger tables* y permite hacer verificación cruzada.
 - ◆ Si A consulta a B, y B le da una ruta falsa, A puede notar inconsistencias con otros nodos.
 - ◆ A pregunta también a C y compara.
- ◆ El atacante no puede insertar nodos intermediarios en todas las rutas sin ser detectado.

Distributed Hash Table (DHT) - Chord - Seguridad

Chord hereda los aspectos de seguridad de una DHT.

2. Nodos no pueden elegir qué datos almacenar

sucessor(k) definido según hash

- ◆ Si quiero almacenar una clave k , todos los nodos saben que debe buscar en $\text{sucessor}(k)$.
- ◆ El atacante no puede elegir fácilmente ser el nodo responsable de la clave k , a menos que pueda controlar su ID, lo que implica romper el *hash*.

Distributed Hash Table (DHT) - Chord - Seguridad

Chord hereda los aspectos de seguridad de una DHT.

3. La red se adapta a nodos que entran/salen

Redundancia de información y *Churn* eficiente de implementar.

- ◆ Normalmente, las *keys* se replican en $\text{sucesor}(k)$, $\text{sucesor}(\text{sucesor}(k))$, etc.
- ◆ Además, ante la llegada o salida de nodos, solo implica actualizar la *finger table* de algunos nodos.
- ◆ Si se detecta un nodo malicioso, es fácil eliminarlo de las *finger table* para ignorarlo y no perder la información que este almacenaba.

Distributed Hash Table (DHT) - Otros ejemplos

Pastry

- ◆ Usa prefijos comunes en las IDs para enrutar.
- ◆ Cada salto lleva a un nodo cuyo prefijo comparte más dígitos con el destino.
- ◆ Similar a un cartero que se acerca paso a paso según coincidencia del código postal.
- ◆ Ejemplo - Buscar (4444)
 - ◆ 4XXX → 44XX → 444X → 4444

Distributed Hash Table (DHT) - Otros ejemplos

Pastry

- ◆ Usa prefijos comunes en las IDs para enrutar.
- ◆ Cada salto lleva a un nodo cuyo prefijo comparte más dígitos con el destino.
- ◆ Similar a un cartero que se acerca paso a paso según coincidencia del código postal.
- ◆ Ejemplo - Buscar (4444)
 - ◆ 4XXX → 44XX → 444X → 4444

Kademlia

- ◆ Usa XOR para medir distancia entre nodos.
- ◆ Cada salto lleva a un nodo que comparte más dígitos, en cualquier posición, con el destino.
- ◆ Como un juego donde se pregunta a quienes tienen números más parecidos al objetivo.
- ◆ Ejemplo - Buscar (4444)
 - ◆ 4XX4 → 44X4 → 4444

Poniendo a prueba lo que hemos aprendido

¿Cual de las siguientes afirmaciones es **correcta** respecto a las redes P2P estructuradas?

- a. En este tipo de red, si un nodo quiere compartir un archivo, tiene libertad de guardarlo en su servidor o entregarlo a otro nodo.
- b. En la red *chord* con 10 nodos, el peor caso de búsqueda es que un nodo le consulte a los 9 restantes nodos.
- c. En una red de 10 nodos, donde ya no entran o salen nodos, usar *hash* tradicional es igual de eficiente que usar *hash* consistente para determinar qué nodo guarda cada información.
- d. Un atacante puede incluir nodos en posiciones específicas de la red para lograr un ataque eclipse sin dificultad.
- e. Ninguna de las anteriores es correcta.

Poniendo a prueba lo que hemos aprendido

¿Cual de las siguientes afirmaciones es **correcta** respecto a las redes P2P estructuradas?

- a. En este tipo de red, si un nodo quiere compartir un archivo, tiene libertad de guardarlo en su servidor o entregarlo a otro nodo.
- b. En la red *chord* con 10 nodos, el peor caso de búsqueda es que un nodo le consulte a los 9 restantes nodos.
- c. En una red de 10 nodos, donde ya no entran o salen nodos, usar *hash* tradicional es igual de eficiente que usar *hash* consistente para determinar qué nodo guarda cada información.
- d. Un atacante puede incluir nodos en posiciones específicas de la red para lograr un ataque eclipse sin dificultad.
- e. Ninguna de las anteriores es correcta.

Próximos eventos

Próxima clase

- ◆ Miércoles de convivencia.

Evaluación

- ◆ Última actividad **obligatoria**: Control de 2 preguntas sobre lo visto hoy.
- ◆ Hoy finaliza plazo para entregar informe y mañana el video.
- ◆ Mañana les indicaré qué informe leer, tienen hasta el domingo para responder.

IIC2523

Sistemas Distribuidos

— Hernán F. Valdivieso López —
(2026 - 1 / Clase 22)
